Reliability-Aware RAG for Turkish Legal QA — Final Report

CS 455 — Large Language Models, Spring 2025/2026

Mehmet Onur Sönmez (32278), Efe Sözer (31012), Mitat Mehmet Erşeker (31258)

Abstract

We set out to build a Turkish legal question-answering system that does not just give an answer, but also shows *where the answer comes from* and *how much to trust it*. Our pipeline retrieves passages from a mix of legal QA explanations and actual statute articles, generates a short grounded answer with citations, and then runs a verifier that labels each claim as `supported`, `partial`, `unsupported`, `insufficient`, `risk`, or `error`. A confidence gate lets the system say "I don't have enough to answer this" instead of guessing.

The most useful thing we learned was about *evaluation*, not the model. A strict "did you retrieve the exact statute article?" metric makes the system look much worse than it is, because in our corpus a question is often answered by a QA passage that quotes the article rather than by the article itself. So we report two numbers side by side. On a 36-question manual set, strict Recall@8 is 0.28 while answer-support Recall@8 is 0.42; turning on the reranker raises these to 0.33 and 0.44, and helps the *ordering* even more (MRR 0.15→0.20 and 0.20→0.26). We are honest about what is still weak: our verifier's exact-match agreement with our own labels is modest and very label-sensitive (we report it as a legacy 35-item diagnostic, not a headline), and a full numeric answer- quality comparison is still future work. Section 4 walks through the four bugs we found and fixed along the way.

1. Introduction

Turkish legal text is hard to read if you are not a lawyer: it is long, formal, and spread across QA pages, explainer articles, and the statutes themselves. A language model can summarize all of this very fluently but it can also state something confidently that simply is not in the sources, which is dangerous when someone might act on a legal answer. Retrieval-Augmented Generation (RAG) helps, because it makes the model answer from retrieved passages, but retrieval can miss and the model can still drift.

Our goal was a system that takes a Turkish legal question and gives back a short, grounded answer *together with* the evidence it used and a clear warning when the answer is only partly supported, unsupported, or risky to treat as advice. Compared with our original proposal, here is what we actually delivered:

- a working three-mode system (LLM-only, RAG, and RAG+Verifier) with a React/FastAPI demo;
- a corpus that we had to extend exactly as our proposal's backup plan predicted from QA-only passages to real article-level statute text; and

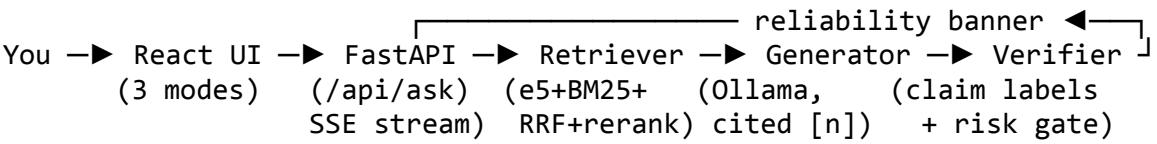
- an evaluation that we tried hard to keep honest, including the parts that did not work.

In short, our contributions are a hybrid retriever (dense e5 + BM25 with reciprocal-rank fusion and a carefully blended reranker), a reliability layer (a claim-level verifier plus a deterministic risk gate and a confidence gate), and a small methodological idea we think is the most interesting result: telling apart *strict* statute-citation recall from *answer-support* recall, with a transparent rule for when an alternative passage counts.

2. System design

2.1 The big picture

Here is the path a question takes through the system:



The backend has a normal `POST /api/ask` endpoint and a streaming `POST /api/ask/stream` one (so the demo can show the answer appearing token by token), plus controls to switch the local model. Everything is driven by one YAML config file (configs/default.yaml), so we could change the embedding model, top-*k*, the reranker, or the thresholds without touching code which saved us a lot of time during experiments.

2.2 The data

Our main source is `OrionCAF/turkish_law_qa_dataset` (Hugging Face, Apache-2.0), about 18,305 Turkish legal QA pairs. We clean each entry (Unicode normalization, HTML stripping, whitespace fixes), drop answers that are empty or shorter than 60 characters, and remove near-duplicate questions. That leaves **17,916** QA passages, which we store as "`Soru: … Açıklama: …`" so that matching the question also pulls in the explanation.

Early on we hit the problem our proposal had warned us about: this dataset contains *explanations*, not the statutes themselves. So a question about, say, who is liable when your dog injures someone (Türk Borçlar Kanunu m. 67) had nothing to ground against the small model would make something up, and the bigger model would (correctly) refuse. To fix this we scraped article-level statute text for 25 core codes (TBK, TMK, TCK, İş K., TTK, HMK, CMK, Anayasa, İİK, plus tax, consumer, traffic, and others) from the public `mevzuat.gov.tr` PDFs and cleaned out the page-heading noise. Scraping gave us 6,490 article rows; after dropping 156 that were empty or just artifacts after cleaning, **6,334** articles made it into the index, each carrying `law_code`, `law_number`, `article_number`, `source_dataset`, and `source_url`. When the live PDF endpoint was flaky we fell back to a public mirror (`muhammetakkurt/mevzuat-gov-dataset`, MIT).

One caveat worth stating up front: we scraped and indexed this in mid-2026. Laws change, and our snapshot will not reflect later amendments, so the `source_url` should always be checked before relying on an article.

We also set aside 50 records (with their source passage as the gold answer) for a self-retrieval smoke test.

Table 1. What is in the index.

Source	Passages	License / origin
OrionCAF QA passages	17,916	Apache-2.0 (Hugging Face)
Indexed statute articles	6,334	Public mevzuat.gov.tr (25 codes; 6,490 scraped, 156 removed)
Total indexed	**24,250**	—
Self-retrieval smoke-test set	50	Seed split of QA corpus

2.3 Retrieval

We embed every passage with `intfloat/multilingual-e5-base` (using the required `query:` / `passage:` prefixes) and store the vectors in a FAISS index over normalized vectors, so the search is cosine similarity. On top of that we added a few pieces that each earned their place:

- **A keyword (BM25) index** alongside the dense one, because dense search alone sometimes misses exact legal wording. It tokenizes Turkish-normalized text into alphanumeric tokens of length ≥ 3, which catches terms like "kıdem", "tazminat", or code abbreviations like "tbk". One honest detail: because the tokenizer throws away tokens shorter than 3 characters, a bare reference like "m. 67" is *not* something BM25 can match directly that kind of exact-article hit comes from our query expansion and a per-passage `match_terms` field instead. We combine the dense and keyword rankings with reciprocal-rank fusion (RRF, k=60).
- **Query expansion**, a small hand-built map that turns everyday phrasing into the legal terms that actually appear in statutes (for example expanding an early-lease-termination question toward "Türk Borçlar Kanunu madde 325").
- **A cross-encoder reranker** (`BAAI/bge-reranker-v2-m3`). We learned the hard way (Section 4) not to let it simply overwrite the retrieval score. Instead we *blend*: the reranker's raw scores are min-max normalized to [0,1] across the candidate pool, the retrieval score is clamped to [0,1], and the final score is 0.70·(retrieval) + 0.30·(reranker). We also keep any passage whose retrieval score is ≥ 0.90 from being pushed down. The 0.70/0.30 split is deliberate: we want the reranker to settle close calls, not to override a clearly strong match.
- **A confidence gate.** If the best retrieval score is under 0.75, or the top three average under 0.70, the system refuses and shows the passages it found rather than dressing up a weak match as a confident answer.

2.4 Generation

Answers are generated by a local Ollama model by default (with the Hugging Face Inference API as a backup). We use a compact 7–9B model locally and tried larger 27B/31B models on a higher-memory GPU machine. The system prompt is strict: answer only from the sources, cite them as `[n]`, refuse if the context is not enough, and never give case-specific legal advice. The LLM-only mode uses the same model with no sources attached, which lets us see exactly what retrieval adds.

2.5 Verification

This is the part we care about most. After an answer is generated:

- A **claim splitter** breaks it into sentence-level claims (carefully, so it does not split on abbreviations like "m." or "md.").
- A single **verifier call** checks every claim against the retrieved passages and returns JSON: for each claim a `status`, the `source_ids` it relied on, and a short reason.
- A **risk gate** then overrides any claim to `risk` if it matches a pattern for a money amount, an imperative ("dava açın"), or absolute certainty ("kesinlikle"). This *guarantees* we catch those specific patterns, but it is pattern-matching, not understanding so it can miss reworded risk (false negatives) or trip on harmless wording (false positives).
- A **citation check** only downgrades a claim when the verifier finds no supporting source or the answer cites a conflicting one; a simply missing `[n]` is treated as a tidiness note, not a real problem.
- An **aggregator** rolls the per-claim verdicts up into one overall verdict, and importantly reports a verifier crash as `error` rather than silently calling it "insufficient."

The demo shows all of this: three modes (A. LLM-only, B. RAG, C. RAG+Verifier), a streaming answer with citations, the source list, a reliability banner, a card to swap local models, and a disclaimer that never goes away.

3. Evaluation

3.1 How we tested

We used two question sets:

- A **self-retrieval smoke test** (50 questions) where the gold passage is the one the question came from. Since that passage is still in the index, this basically checks that search works at all it is a sanity check, not proof of quality.
- A **manual set** (51 questions) we wrote by hand to be realistic: paraphrased questions, ambiguous ones, ones with no answer in the corpus, future-law questions, risky ones, and many that target a specific statute. Each one has a type, an expected verdict, an optional gold passage, and a one-line note, and we wrote a rubric (evaluation/annotations/RUBRIC.md) so we were labeling consistently. Of the 51, 36 have a gold passage and are scored for retrieval.

One thing to keep straight: the retrieval numbers below use the current 51-question set (36 scored), but the verifier number uses an *earlier* 35-question version, so we treat the verifier result as a rough diagnostic rather than something directly comparable (Section 3.4).

3.2 Retrieval results

We report two numbers that should never be mixed up:

- **strict** — did we retrieve the one annotated statute article? (precise but harsh)
- **answer-support** — did we retrieve *any* acceptable passage, meaning the article itself *or* a passage that explicitly quotes it? We only add an alternative when a passage literally names the article, so we are not just loosening the metric to look good.

On the smoke test, retrieval is basically perfect (Recall@3/5 = 1.0, MRR = 0.98, n=50), which is expected and not very meaningful. The manual set is where the real story is (Table 2).

Table 2. Manual-set retrieval (n = 36), reranker off vs. on.

Scoring mode	Rerank	Recall@3	Recall@5	Recall@8	MRR
Strict (single statute gold)	off	0.19	0.22	0.28	0.15
Strict (single statute gold)	on	0.22	0.31	0.33	0.20
Answer-support (cited-article union)	off	0.33	0.36	0.42	0.20
Answer-support (cited-article union)	on	0.33	0.42	0.44	0.26

Two things stood out to us. First, the strict number really does undersell the system: without reranking, only 10 of 36 questions retrieve the exact article in the top 8 (strict = 0.28), but 15 retrieve something that actually answers the question (answer-support = 0.42); with reranking those become 12 (0.33) and 16 (0.44). Second, the reranker helps the *ranking* more than the raw recall the answer-support ceiling barely moves (0.42→0.44), but MRR jumps (0.20→0.26) and the exact-article (strict) recall improves the most, because the reranker finally pulls the right article up once we stopped it from dropping good matches (Section 4). Either way, these are retrieval signals, not a measure of whether the final answer is correct.

3.3 Comparing the three answer modes

We will be straight about this: we did not finish a full numeric scoring of correctness and faithfulness across all three modes that needs careful human (or LLM-judge) grading and is our biggest piece of unfinished evaluation. We did not want to invent scores, so instead we report what each mode concretely does on the same questions, which anyone can reproduce in the demo (Table 3).

Table 3. What each mode actually produces.

Property	A. LLM-only	B. RAG	C. RAG+Verifier
Uses retrieved sources	No	Yes	Yes
Inline `[n]` citations	No	Yes	Yes
Refuses on low coverage (confidence gate)	No	Yes	Yes
Per-claim `supported/partial/...` verdicts	No	No	Yes
Deterministic legal-advice risk flag	No	No	Yes
Surfaces partial/unsupported claims to the user	No	No	Yes

A concrete example we kept coming back to: “Köpeğim birine zarar verirse tazminatı kimden isterim?” (who pays if my dog hurts someone?). After our score-blending fix, retrieval puts the animal-keeper liability article (TBK m. 67) at the top. LLM-only answers smoothly with no source; RAG answers and cites the article; RAG+Verifier additionally marks the main claim `supported` and, if the answer happens to state a compensation amount, the risk gate flags it as advice-risk. That is the difference the reliability layer is meant to make.

3.4 Verifier (a legacy 35-item diagnostic)

On the earlier 35-question version of the set, the verifier’s overall verdict matched our expected label 18 times out of 35 (0.51; an earlier run got 0.54). Because the current set has 51 questions, we treat this as a diagnostic we are showing for transparency, not a headline result. The confusion matrix (Table 4) tells us more than the single number does.

Table 4. Verifier confusion matrix (legacy set, n = 35). Rows = expected, columns = predicted.

Predicted → Expected ↓	supported	partial	unsupported	insufficient	Total
supported	15	3	2	0	20
partial	4	3	4	0	11
unsupported	0	0	0	0	0
insufficient	1	1	2	0	4
Total predicted	20	7	8	0	35

Reading it, the mistakes are clearly *label-sensitive* and bunch up on the fuzzy `partial` / `unsupported` / `insufficient` boundary: the system never once says `insufficient` (0 vs. 4 expected) and says `unsupported` 8 times even though we labeled nothing that way, while it gets `supported` mostly right (15 of 20). So we read 0.51 as inconclusive about real quality rather than a clean failure, for three reasons we can point at but have not fully measured: (i) some of our gold labels are stale a labour question we marked `insufficient` before we indexed İş Kanunu is arguably `supported` now; (ii) the line between `partial` and `supported` is genuinely subjective; and (iii) we sometimes labeled `risk` at the question level, but the system only fires risk at the answer level. We tried to re-run the verifier on the full 51-question set with a larger 27B model on a higher-memory GPU machine, but it stalled on the long verifier prompts and never finished, so rather than report half a result we left it out and listed it as future work.

The risk gate is the steadier part: it flags every answer matching its money/imperative/certainty patterns no matter what the model thinks but, again, that is a promise about *those patterns*, not a guarantee of catching every unsafe phrasing.

3.5 Ablations

- **Reranker off vs. on** (Table 2): once the score-blending fix was in, answer-support Recall@8 went 0.42→0.44 and MRR 0.20→0.26, with most of the gain in ranking quality and exact-article recall.
- **top-*k* in {3, 5, 8}**: bigger *k* raises recall but waters down the context we hand to the generator, so we settled on *k* = 8.
- **Embedding size**: swapping e5-base for e5-large only nudged strict recall to about 0.30, which told us the embedder was not the thing holding this metric back.

4. What went wrong (and how we fixed it)

The assignment specifically asked us to write about what did not work, and we had plenty of material. Here are the four bugs that shaped the final design.

(1) The reranker was throwing away the right answers. When we first turned reranking on, it *replaced* the retrieval score outright. For the dog / m. 67 question, the dense retriever ranked the exact article first (0.97), but the reranker scored it near zero and buried it,

simply because the article does not literally mention a money amount. **Fix:** blend 0.70 retrieval + 0.30 reranker and protect strong matches (Section 2.3), so the reranker reorders without discarding clearly-correct hits.

(2) The confidence gate was reading the wrong scale. Our thresholds (0.75 / 0.70) were tuned for e5 cosine scores, but with reranking on, the gate was suddenly looking at the reranker's 0–1 scale and refusing perfectly good questions. **Fix:** check confidence on the original retrieval scores *before* reranking. The dog / m. 67 case went from "not enough sources" to a proper grounded answer.

(3) The verifier was too strict about citations. A claim the verifier had actually grounded in a real source was getting downgraded from `supported` to `partial` just because the sentence was missing its `[n]` marker. **Fix:** only downgrade on a real citation conflict or a genuine lack of support; treat a missing marker as a note, not a failure.

(4) The metric, not the model, was hiding the truth but only partly. Our low strict recall (0.28) is driven a lot by how strict the metric is and by what is in the corpus, not just by model power a bigger embedder only moved it to about 0.30. That is why we added the answer-support metric, which raised measured Recall@8 to 0.42. We are careful not to over-claim here, though: answer-support Recall@8 is still only 0.42–0.44, so retrieval ranking and corpus coverage are genuinely real limits too (Section 5). To keep the metric honest we only add an alternative gold when a passage names the article outright, and we marked a question whose statute we never indexed (Kabahatler Kanunu) as out-of-corpus instead of forcing a match.

5. Limitations

- **Coverage.** Even with 25 codes, we do not have case law, amendments, secondary legislation, or several referenced codes (Kabahatler, İSG came up in questions but are not indexed). The answer-support Recall@8 of 0.42–0.44 is a reminder that coverage and ranking not only metric strictness still hold us back. This is a curated slice, not all of Turkish law.
- **PDF noise.** Statute text comes from PDFs; our cleaner helps, but some artifacts survive, and we dropped 156 rows that were too noisy to index.
- **The verifier is just another model.** By default it is not independent of the generator (the same model can do both), so it inherits the same blind spots.
- **Small evaluation.** Retrieval (n=36) and the verifier diagnostic (n=35, legacy) are small, so one label change moves the numbers by a couple of points. Some gold labels were drafted with AI help and still need a full human pass, and the numeric answer-quality table (Section 3.3) is not done yet.
- **Compute.** Bigger local models read Turkish better but are slower and heavier; our default sticks to a compact model that runs on a laptop GPU.

6. Ethics and educational use

We mean it when we say this is for learning, not legal advice. Three things back that up: a disclaimer that stays on screen and tells people to see a real lawyer; a system prompt that

forbids case-specific advice and forces a refusal when the context is thin; and the risk gate that flags money, imperatives, and certainty even when the model sounds sure (with the limits we noted in Section 2.5). All of our data is public and used under its license (Apache-2.0 / MIT / public statute text), and we use no private or personal data. The statute text is public legislation from `mevzuat.gov.tr`, and we keep `source_url` provenance. Full disclaimers are in docs/ethics/disclaimer.md.

7. What we would do next

- Finish the numeric LLM-only / RAG / RAG+Verifier comparison (correctness, faithfulness, refusal, safety).
- Re-label the manual set against the bigger corpus and finish the verifier run on all 51 questions, with shorter prompts so the larger model does not stall.
- Add more of the legal corpus (the missing codes, ideally case law) and check for freshness.
- Make the verifier independent of the generator, and replace the sentence splitter with a smarter claim-by-claim splitter.
- Try fine-tuning a Turkish reranker on our own data.

8. Reproducibility and compute

The repo has a README with setup and run steps. `python scripts/build_index.py` does load → clean → embed → FAISS; `python scripts/serve.py` runs the demo; `python scripts/run_eval.py` reproduces the retrieval/verifier numbers and ablations; and `scripts/debug_retrieval.py` prints per-question diagnostics. We also wrote **116 tests** covering data cleaning, claim splitting, the verifier logic, retrieval fusion, and the metric. Indexing and the retrieval/rerank evaluation run on modest hardware (a 6 GB laptop GPU, or even CPU); only the larger 27B/31B generation models needed a higher-memory GPU machine. We pinned our dependencies (especially `transformers` and `sentence-transformers`) so the setup is reproducible.

9. Who did what

- **Erşeker** — statute ingestion and getting it into the corpus, the verifier reliability fixes, and the retrieval metric (strict vs. answer-support) plus the eval-set annotation.
- **Sözer** — the `mevzuat.gov.tr` scraping and cleaning, and the hybrid BM25 + FAISS retrieval.
- **Sönmez** — query expansion and RRF ranking, the reranker stability / score-blending fix, the statute import tooling, and documentation.

We all worked together on integration, the demo, and this report.

10. How we used AI assistants

As the academic-integrity policy asks, here is how we used an AI coding assistant: it helped us debug the retrieval / verifier / metric code, work out the four bugs in Section 4, draft candidate eval-set labels under our own explicit acceptance rule (which we then reviewed),

and draft prose for this report that we edited and checked. Every number in this report came from our own evaluation scripts on our own data.

11. References

1. P. Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* arXiv:2005.11401.
2. L. Wang et al. (2024). *Multilingual E5 Text Embeddings: A Technical Report.* arXiv:2402.05672.
3. G. Cormack, C. Clarke, S. Büttcher (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.* SIGIR.
4. N. Reimers, I. Gurevych (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP.
5. BAAI. *bge-reranker-v2-m3.* Hugging Face model card.
6. S. Robertson, H. Zaragoza (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in IR.
7. J. Johnson, M. Douze, H. Jégou (2019). *Billion-scale similarity search with GPUs (FAISS).* IEEE Big Data.
8. OrionCAF. *Turkish Law QA Dataset.* Hugging Face.
9. *muhammetakkurt/mevzuat-gov-dataset* (public `mevzuat.gov.tr` mirror, MIT). Hugging Face.